6.5 Handling preflight requests

In the beginning of the chapter we talked about simple CORS requests. But if your server handles anything beyond simple `GET`s or `POST`s, you also need to respond to preflight requests. Chapter 4 introduced the basics of handling preflight requests. As covered in that chapter, preflight requests are a necessary part of the CORS flow, and help protect servers from unsupported requests. But a preflight request can be a performance hit, because it requires two HTTP requests. This can especially be an issue on resource-constrained systems, like mobile phones. This section investigates strategies for both handling and reducing preflight requests.

6.5.1 Whitelisting request methods and headers

The preflight request is issued to verify that the server allows certain HTTP methods and headers. In the blogging sample, a preflight request is issued before a post can be deleted to ensure that HTTP `DELETE` requests are allowed.

Earlier in this chapter we used a whitelist to validate the origin header. You can use a similar whitelist to specify valid HTTP methods and headers. The code in the following listing updates the blogging sample application in app.js with two new options for configuring preflight requests, called `allowMethods` and `allowHeaders`.

Listing 6.8 Allowing HTTP methods and headers

```
var corsOptions = {
  allowOrigin: createWhitelistValidator(originWhitelist),
  allowCredentials: true,
  shortCircuit: true,
  allowMethods: ['GET', 'DELETE'],                          // Configuration options for allowed HTTP methods and headers
  allowHeaders: ['Timezone-Offset', 'Sample-Source']
};

var handleCors = function(options) {
  return function(req, res, next) {
    ...
    if (isPreflight(req)) {
      if (options.allowMethods) {
        res.set('Access-Control-Allow-Methods',             // Sets preflight-specific headers to configuration values
            options.allowMethods.join(','));
      }
      if (options.allowHeaders) {
        res.set('Access-Control-Allow-Headers',
            options.allowHeaders.join(','));
      }
```

The `allowMethods` and `allowHeaders` options contain the whitelisted HTTP methods and headers that are allowed by the server. The server uses these values to set the corresponding preflight headers on the response. Now that the code is configured to set the preflight headers, the far more interesting question is: what values do you choose for the whitelist?